16-FFI-AND-GPU-SAFETY.md — LibRaw, wgpu and ONNX without crashes

📎 Rust guarantees memory safety right up to the moment you call C. AURA calls C for RAW decoding, GPU drivers and inference runtimes — the three places a wedding run actually dies. This file makes those boundaries defensive by construction.

1. The unsafe boundary policy

- All `unsafe` lives in dedicated `-sys` wrapper crates: `aura-libraw-sys`, `aura-ort-sys`. No `unsafe` anywhere else in the workspace — enforced by `#![forbid(unsafe_code)]` in every other crate.
- Every `unsafe` block carries a `// SAFETY:` comment stating the invariant being upheld. A block without one fails review automatically.
- Wrapper crates expose only safe, owned Rust types. No raw pointer ever escapes a wrapper.
- Every C resource is owned by an RAII type with a `Drop` impl. No manual free calls in application code.

2. LibRaw hardening

RAW decoding is the single most hostile input surface in the product: attacker-shaped or simply broken files from dozens of camera bodies.

Risk	Mitigation
Segfault or abort inside C on malformed input	Decode in a **separate short-lived worker process**, one file at a time; a crash kills the worker, not the app. The parent quarantines the file and continues.
Unbounded allocation from a forged header	Validate declared dimensions against a sanity ceiling (currently 250 MP) and against file size before allocating.
Infinite loop in a decoder path	Per-file wall-clock deadline (default 20 s); worker is killed on expiry, `AURA-RAW-2008`.
Buffer over-read on truncated files	Length and magic-byte validation before handing bytes to C; fuzzed corpus in CI.
Library version drift changing colours	LibRaw version + build flags recorded in the render fingerprint; upgrading it requires an ADR and a golden regeneration.
Thread-unsafe global state	One `libraw_data_t` per worker; never shared, never reused across files.

Frozen wrapper shape

```
// crates/aura-libraw-sys/src/lib.rs — FROZEN CONTRACT v1
pub struct RawImage { /* owned pixel buffer, no C pointers */
}

pub struct DecodeLimits {
    pub max_megapixels: u32,   // 250
    pub max_alloc_bytes: u64,  // 2 GiB
    pub deadline_ms: u32,      // 20_000
}

/// Decodes in an isolated worker process. Never panics; neve
r aborts the caller.
pub fn decode_isolated(path: &Path, limits: DecodeLimits) ->
AuraResult<RawImage>;
```

Fuzzing is mandatory: `cargo fuzz` target over a seed corpus of every supported format plus mutated variants, run nightly for at least 30 minutes. New crashes become fixtures in `wedding-c-disaster`.

3. GPU (wgpu) hardening

Risk	Mitigation
Device lost / driver reset (TDR)	Register a device-lost callback; recreate the device once; on second loss fall back to the CPU develop path and emit `AURA-GPU-4001` at `Degraded`.
Watchdog timeout on long kernels	Size every submission to ≤ 2 s of work; tile large images; never one giant dispatch.
VRAM exhaustion	Probe available VRAM at startup; a tile-size solver picks dimensions; on allocation failure, halve tile size and retry down to a floor, then fall back to CPU.
Shader compile failure on a specific driver	Compile and validate the entire shader bundle at startup, not lazily mid-run; failure disables GPU path before any work is claimed.
Buffer alignment / stride mistakes	All buffer sizes computed by a single helper honouring `COPY_BYTES_PER_ROW_ALIGNMENT`; unit-tested against odd dimensions.
Out-of-bounds reads in WGSL	Clamp all texture coordinates in-shader; enable wgpu validation in debug and CI builds.
Resource leaks across cancelled runs	All GPU resources owned by a per-run arena that is dropped wholesale on cancel; test asserts zero live buffers after 200 cancel cycles.
Precision drift between backends	No `f16` in colour-critical paths; no fast-math; see `12-DETERMINISM-CONTRACT.md`.

Every GPU stage must have a CPU reference implementation. It is the fallback, the oracle for correctness tests, and the D0 baseline. A GPU stage without a CPU twin cannot pass phase exit.

4. ONNX Runtime hardening

- **Verify before load** — signature + SHA-256 check on the model package; refuse unsigned (`AURA-ML-5001`). Models are treated as untrusted binaries.
- **Assert the chosen EP** — after session creation, read back the active provider and compare to intent. A silent CPU fallback is a `Degraded` event, never invisible (F-18).

- **Pin session options** — deterministic algorithm selection, fixed intra/inter-op thread counts, arena limits set from the VRAM budget.
- **Validate I/O shapes** — compare the runtime's reported shapes to `models/<name>/signature.json` before the first inference; mismatch raises `AURA-ML-5007` and disables the model rather than producing garbage.
- **Output sanity** — every inference result is checked for NaN/Inf and for range; violations quarantine the item rather than propagating into pixels.
- **Session lifetime** — sessions are pooled and reused; never created per photo. Pool size derives from the VRAM budget, and creation is serialised.

5. Process isolation map

Component	Where it runs	Why
UI	Main process (WebView)	Responsiveness
Orchestrator, catalog, develop	Main process (Rust)	Shared state, speed
RAW decode	Worker process pool	Hostile input; crash containment
Generative cleanup	Worker process	Large models, isolation, easy VRAM reclaim
Crash reporter	Separate process	Must survive the crash it reports (F-38)

Worker protocol: length-prefixed messages over stdio, a version handshake on start, a heartbeat, and a hard kill on deadline. Workers are stateless — killing one is always safe.

6. Test plan

- **Fuzz** — nightly `cargo fuzz` on the RAW decoder; zero new crashes required to ship.
- **Crash containment** — inject a deliberate segfault in the decode worker mid-run; app must quarantine one file and finish the wedding.

- **Device-lost simulation** — force device loss on Windows (TDR trigger) and macOS; assert single recreate then CPU fallback, and assert identical output within D1 tolerance.
- **VRAM squeeze** — cap VRAM at 2 GB; the tile solver must complete a 61 MP file without OOM.
- **Odd dimensions** — render images with prime-number widths to catch stride bugs.
- **Leak test** — 200 cancel cycles; zero live GPU buffers, zero live worker processes, RSS returns to baseline ± 5 %.
- **Model tamper test** — flip one byte in a model file; loading must refuse with `AURA-ML-5001`.

7. Acceptance criteria

- [ ] `#![forbid(unsafe_code)]` in every crate except the two `-sys` crates.
- [ ] Every `unsafe` block has a `SAFETY:` comment; CI greps for violations.
- [ ] RAW decoding runs out-of-process with limits and deadlines.
- [ ] Every GPU stage has a CPU reference and passes a parity test.
- [ ] Active EP is asserted and surfaced in the UI and telemetry.
- [ ] Fuzz job green with zero uncontained crashes.

8. Claude Code prompt

> Act as `SRG` (Senior Engineer — GPU) for the wgpu sections and `SRC` for the FFI sections of `16-FFI-AND-GPU-SAFETY.md`. Implement `decode_isolated` with the frozen signature, the worker protocol, the tile-size solver, the device-lost fallback ladder, and the EP assertion. Add `#![forbid(unsafe_code)]` to every crate that does not need it and fix the fallout. Then switch hats to `QAL` and implement the crash-containment, device-lost, VRAM-squeeze, odd-dimension and leak tests. Report PASS/FAIL per test with command output. Do not silence a failing test by widening a tolerance.